

SEN381

ASSIGNMENT T 1

Research Foundations for
Software Engineering
Decisions

Chanice Grant (601597)
Megan Heather Wathen (601611)
Milenke Field (578122)

Table of Contents

QUESTION 1 – SOFTWARE ENGINEERING BEYOND PROGRAMMING	2
1.1 Introduction	3
1.2 Software Engineering Beyond Functioning Code.....	3
1.3 Interaction of Software Engineering Concerns	3
1.3.1 Stakeholder Needs and Requirements	3
1.3.2 Scope.....	4
1.3.3 Schedule	4
1.3.4 Cost and Resources.....	4
1.3.5 Quality.....	5
1.3.6 Engineering Risk	5
1.4 Interaction and Trade-offs Between the Concerns	6
1.5 Real-World Software Engineering Example	6
1.5.1 Background.....	6
1.5.2 Engineering Decision, Assumption or Constraint.....	6
1.5.3 Trade-off or Constraint	7
1.5.4 Downstream Consequence	7
1.5.5 Engineering Lesson	7
1.6 Critical Discussion	8
1.7 Conclusion.....	9
QUESTION 2 – THINKING AHEAD: DECISIONS ACROSS THE SOFTWARE LIFECYCLE	9
2.1 Introduction.....	9
2.2 Model Reference Introductory Examples	10
2.2.1 Snapshot of Waterfall Model:	10
2.2.2 Snapshot of Agile Model:.....	10
2.2.3 Snapshot of Incremental Model:.....	10
2.3 Significant Software Engineering Decisions and/or Concerns.....	11
2.3.1 Decision/Concern 1: Microservice or Monolithic Architecture.....	11
2.3.2 Decision/Concern 2: Non-Functional Requirements – Including Cost Along with Scalability, Latency, Reliability, etc.	12
2.3.3 Decision/Concern 3: Migratable Deployment Setup	13
2.3.4 Decision/Concern 4: Method of Database Integration	14
2.3.5 Decision/Concern 5: Audit Logging and Monitoring Strategy	14
2.3.6 Decision/Concern 6: Authentication, Authorization, and Identity Concerns	15
2.4 Decision Horizon Map.....	16
QUESTION 3 – EVIDENCE-BASED TECHNOLOGY STACK DECISIONS.....	17
3.1 Introduction.....	17

Stack A — Java-based web stack	17
Stack B — .NET-based web stack	17
3.2 Overview of the Two Alternatives	18
Stack A: React + Spring Boot + PostgreSQL.....	18
Stack B: React + ASP.NET Core + PostgreSQL.....	18
3.3 Comparative Analysis	18
3.4 Weighted Decision Matrix	20
3.5 Justification of the Weights	20
3.6 Evidence-Based Interpretation of the Matrix	21
3.7 Limitations of the Weighted Decision Matrix	22
3.8 Critical Question	22
Why is the highest-scoring or most popular technology stack not automatically the correct technology for every software project?	22
QUESTION 4 – RESPONSIBLE AI IN SOFTWARE ENGINEERING	23
4.1 Introduction.....	23
4.2 Three Valuable Uses of Generative AI	23
1. Requirements analysis and documentation	23
2. Coding and software design assistance	24
3. Testing, review and technical documentation	24
4.3 Three Significant Risks and Limitations	25
4.4 Human Verification and Control Process	26
4.5 Accountability	27
If an engineer accepts AI-generated output that later introduces a defect, vulnerability or incorrect engineering decision, who remains accountable and why?.....	27
4.6 Conclusion.....	28
REFERENCE LIST	28
Bibliography	28
APPENDIX A – AI RESEARCH & VERIFICATION RECORD	32

QUESTION 1 – SOFTWARE ENGINEERING BEYOND PROGRAMMING

1.1 Introduction

One popular misconception about software development is that it's mainly about coding and creating software that works. But a system that does what it is supposed to do doesn't automatically make it a successful software engineering product. Software engineering recognises that the software system life cycle considers the requirements of the stakeholders, the scope of the software system, the time available for the project, the resources available, and the quality expectations and engineering risks. These concerns are all interdependent, so a decision made in one area can have constraints or consequences in another area of the project. This section, then, will explore the role of those aspects of software development that cannot be reduced to programming, and how they impact each other and the resulting project.

1.2 Software Engineering Beyond Functioning Code

The work of software engineering doesn't end once you've chosen a language and written valid code that runs without error. While implementation is a key development activity, the broader engineering process must be well managed to ensure project success. (Nizam, 2022) shows that failure in software projects can manifest itself in a series of interconnected phases and behaviours, not only of the technical team but also of non-technical actors. This means that overall project success can't just be gauged by code that is working.

The specification, development, evaluation and maintenance of software during its lifecycle must therefore be considered from a more general engineering point of view. Failure analysis is crucial as it can uncover the root cause of failure, the nature of failure, the lifecycle of failure, and other knowledge gained from failure analysis has been applied in software evolution, defect detection and repair (Amusuo, et al., 2022), and many other fields. Therefore, in addition to the focus on implementation, one needs to also have an eye for the decisions and conditions that go into successful Software Engineering. Stakeholder needs, requirements, scope, schedule, resources, quality, and risk all impact whether a software product is suited and viable after its code first performs.

1.3 Interaction of Software Engineering Concerns

1.3.1 Stakeholder Needs and Requirements

Stakeholder needs and requirements become the basis for the goals and objectives which a software system will have to achieve and thus affect many of the engineering decisions that follow. Requirements engineering is about the understanding and communication of these needs to be able to turn them into clear expectations for the software. This process is not entirely technical, since good communication among stakeholders and development team is essential to build a common understanding of the desired system.

This interaction is instrumental as shown by (Hasnain, et al., 2023) whose empirical investigation of 104 software practitioners, identified that misinterpretation and miscommunication of requirements were important factors leading to the failure of software-systems. When requirements are not fully understood or reflected in the software, developers could end up creating something that works technically but doesn't meet the goals of the software. The implications of these misconceptions can be a change in the scope, schedule, resources and quality of the project if the misunderstandings are corrected

later. Requirements of stakeholders should thus be considered an engineering problem that permeates requirement engineering decisions, and not just as data gathered prior to programming.

1.3.2 Scope

Project scope is the outline of a software project and helps to determine its functionality and work. It has a close association to stakeholder requirements as an increase or change in requirements will impact the amount of work to be done. Because requirements may change during development, often adding, removing or modifying them, this can lead to changes in the project scope. Requirements volatility has been found to have a meaningful impact on project cost, schedule and software quality, thereby proving that scope cannot be managed separately from any other engineering issues.

If the scope of the project is increased but there is no time, resource or budget increase, it may entail the development team to perform additional tasks within the same limits. This can add rework, strain on schedule and resources, and quality and engineering risks. On the other hand, if scope is managed too strictly, then the project will not be able to adapt to legitimate changes in the needs of stakeholders. Effective Software Engineering, therefore, demands a view of scope changes as engineering trade-offs, where the worth of the scope change is judged against its implications for cost, schedule, quality and risk.

1.3.3 Schedule

The project schedule determines the amount of time available to perform software engineering activities, and is closely tied to requirements, scope and available resources. If the needs change or the scope of the project grows, more analysis, development, testing, and rework may be needed. If the original completion date is the same, the development team is responsible for adding this work to the current development time frame. The significance of requirement volatility lies in the fact that requirements may change during the development process, which can impact the project's timeline and budget (Pasham, 2024).

Pressures for schedule can also impact engineering choices regarding quality and risk. It is possible that in a limited time, teams will cut out activities like testing, documentation, code review, or requirements validation due to meeting a deadline. While this might offer a short-term scheduling advantage, it can also make it more likely that defects, misunderstandings or maintainability issues go unresolved. To manage a schedule, you should not just ensure that the development is finished on time, but also balance the time-based requirements with the scope, resources, quality, and risks that are introduced by the timing of your decisions.

1.3.4 Cost and Resources

The capacity to provide the software within the scope and schedule as agreed depends on the cost and resources. The resources that are necessary for software development, and the time involved in activities like analysis, implementation, testing and maintenance, as well as other resources, are also integral to the software development process and are as important to the software development process as the money is. As a result, scope or requirements may change, which could result in more work, and possibly more resources. Requirements volatility research illustrates this relationship: that the more requirements change, the more the project costs, schedule and software quality are impacted.

Engineering trade-offs may also be necessary due to resource limitations. If extra funds, staff or time is not available, the team may have to adapt scope, timetables and/or resource

allocations. Trying to stick to the original scope and timeline with not enough resources can put added stress on the development team and can lead to project complications. In failed software projects, for instance, (Nizam, 2022) finds that schedule slippage and disregarded deviations can escalate to worse issues in software projects, such as attempts to catch-up by allocating more resources to the project despite adverse conditions. When working on a constraint, you need to evaluate it in conjunction with scope, schedule, quality, risk, and other areas of the project not just cost.

1.3.5 Quality

Software quality is not just about running software properly, but whether it meets its requirements and is reliable and appropriate for the purpose it was designed for. Quality will thus need to be considered throughout the software life cycle, such as in the requirements analysis, design, development, testing, and maintenance phases. This depends closely on the other engineering concerns discussed above, as this is an integral part of quality.

Requirements and scope changes may impact quality if work previously done has to be changed and rechecked. Studies of the relationship between changing requirements and project cost, schedule and software quality have revealed strong correlations between changing requirements and the other engineering concerns, showing a change in one concern can trigger repercussions in several others. Likewise, the NASA's Software Engineering Handbook mentions that changing requirements can lead to an increase in verification and validation work and higher defect risk because it cascades through requirements, interfaces, and software modules.

Quality may also be put to the test if the engineering effort is restricted due to budget or time pressures. Limiting testing, review and verification, or other activities may be helpful if you have an imminent deadline or limited resources, but it could lead to defects and inconsistencies going undetected in the software. Quality should not be a final inspection that occurs after programming is completed, it is an engineering consideration that needs to be considered, continually, in relation to requirements, scope, schedule, acceptable levels of resources and risk.

1.3.6 Engineering Risk

Engineering risk is the potential for unpredictable events, assumptions or decisions to negatively impact the delivery and operation of a software system. All the above engineering issues can introduce a risk. Requirements that are unclear or evolving can lead to wrong functionality or rework, scope creep can cause strain on the project timeframe and resources, and lack of time or resources can affect activities related to quality. Engineering risk is not a standalone consideration, but evolves over the course of the project, as decisions and constraints interact.

Potential problems need to be identified and addressed before they cause more significant problems that are difficult or expensive to control. (Nizam, 2022) shows that the failure of software project can evolve in a manner connected to each other starting from the warning signs or deviations which can be underestimated or ignored and then moving to more serious software project issues. This is an example of how the engineering team needs to understand and continually assess the effect of changes in requirements, scope, schedule, resources and quality on the overall risk of the project. Risk management can then inform teams of possible consequences, act upon warning signs, and make informed risk-adjusted engineering decisions before they become a factor in project failure.

1.4 Interaction and Trade-offs Between the Concerns

The six Software Engineering concerns are not independent; they are a set of constraints which should be balanced during software construction. The requirements of the stakeholders might change, which could mean that more development and testing is needed to meet the new requirements. With the same schedule and budget, the team will have to determine whether to invest in more resources, cut other functionality, give the project a longer timeline or take on more quality and engineering risk. Requirements volatility research shows this inter-dependency, showing that changes in requirements have substantial impacts on cost, schedule and software quality.

For these interactions, there are engineering compromises to make as it may not be possible to achieve all the conflicting goals at once. A shorter schedule can provide earlier delivery, but less verification and testing time; more quality activities (more time and resources); and less scope may deliver to the schedule and budget but not satisfy some stakeholder needs. Importantly, when a constraint becomes difficult, there is no simple trade-off between sacrificing quality. Rather, other options and their impacts must be reviewed and a reasoned decision made by the team based on stakeholder value, resources available and the acceptable risk. This is an example of why a more holistic view of the project is needed for successful Software Engineering, rather than optimizing one aspect at a time.

1.5 Real-World Software Engineering Example

1.5.1 Background

The Mars Climate Orbiter is an example of a well-documented occurrence in which a software failure is caused by the wider problems with Software Engineering and systems engineering. The spacecraft was launched on 11 December 1998 to learn more about the atmosphere and climate of Mars and to serve as communications for the Mars Polar Lander. The spacecraft arrived on Mars after about nine months and tried to get into orbit on 23 September 1999. There was a loss of communication during orbit-insertion manoeuvre and communication was not restored.

In NASA's next investigation, the trajectory computation was found to be incorrect, due to ground software providing thruster-related data in English units, not the metric units called for by the software interface specification. The cumulative errors on the way there led the spacecraft to come into Mars much lower than planned, thus ending the mission. Importantly, NASA's investigation did not focus solely on the mismatch of units, but on the broader engineering issue. Contributing factors were poor communication between project teams, poor ground software verification and validation, poor staffing and training, and poor systems-engineering process.

1.5.2 Engineering Decision, Assumption or Constraint

One critical engineering assumption for the Mars Climate Orbiter project was the units used when sending data to and from software components. The units for the Angular Momentum Desaturation data provided to the trajectory model had to be metric units for the software interface documentation. The trajectory modelling team thus assumed that the data received does meet this requirement. The ground software, however, that generated the data had

provided thruster performance data only in English measurements instead of the desired metric measurement.

The engineering challenge was therefore not only the fact that there was a wrong unit in the software, but that it was a very big one. Assumptions were made that a requirement for the interface had been properly put in place, but the assumption was not properly tested throughout the software interaction. Later, NASA pinpointed insufficient verification and validation of ground software, communication problems between elements of the project, and problems in the systems engineering process as contributing factors to the failure. This illustrates that it is possible for an apparent small implementation error to become a system level risk if requirements, interfaces, communication and verification are not properly managed.

1.5.3 Trade-off or Constraint

The project of the Mars Climate Orbiter functioned in the context of limited resources and the timeline, which affected the engineering processes related to the mission. The investigation revealed some deficiencies in operations navigation staffing and training, and deficiencies during verification and validation of the ground software. The constraints will have a significant impact as limited personnel and engineering capacity can limit the opportunity for independent review, complete testing and effective investigation of unexpected system behaviour.

This trade-off can be viewed as having enough project cost and time targets but with reduced engineering resources and assurance. The restraints served the mission's short-term goals of meeting mission requirements within resource and time limits but made assumptions and limited the effectiveness of processes that might have identified the software-interface problem. The case, therefore, serves as an illustration that economies of time or resource can lead to increased quality and engineering risk in another part of the system.

1.5.4 Downstream Consequence

The data was misaligned due to a difference in measurement units between English and metric and was then applied to the mission's trajectory calculations for Mars Climate Orbiter. The receiving team did not realize that there was a unit error and thus did not correct it in the navigating process. NASA calculated that the resulting trajectory dropped the spacecraft to an altitude of ~60 km above Mars, instead of the planned altitude of ~150 km. This was lower than the minimum survivable altitude of 85 km and it was never possible to communicate with the spacecraft during Mars orbit insertion.

This changed in effect downstream was far more significant than the initial software interface error. A mismatch in the units used by two software components developed into inaccurate navigation calculations, an incorrect spacecraft trajectory and ultimately the loss of the entire mission. (Prause, et al., 2024) also note the Mars Climate Orbiter, another spaceflight failure that occurred due to a measurement-unit mismatch and other software-engineering issues. The case illustrates the case of a relatively small defect that can have very serious consequences when it crosses the system interfaces without being identified through verification and validation and communication processes.

1.5.5 Engineering Lesson

The Mars Climate Orbiter is a good example of the fact that building functional software code is not enough; controlling requirements, interfaces, communication, verification and

engineering risk throughout the entire system is also essential to successful Software Engineering. The metric-unit requirement was documented but it was assumed that the data that was received from the ground software met that requirement. This inconsistency was not detected due to poor communication, staffing, training and verification and validation. The next lessons NASA learned were increased demands for requirements validation, technical interface verification, independent review of mission critical software, and communication between engineering departments.

The main engineering lesson is thus summarized as:

Assumption that data from the interface met the required units – reduced the detection of the mismatch and led to propagation of incorrect navigation data in the system, which aided in the loss of the spacecraft.

What is evident from the case is that a relatively simple software error can turn into a catastrophic failure of the system when the surrounding engineering processes are not able to identify and control the error. The software engineering process must consider the effects of software technical decisions and assumptions on the whole system, not just whether each software component does what it is supposed to do. This is aligned with the wider systems view suggested by (Matranga, et al., 2024) where communication and collaboration between systems is key to uncovering and controlling risks between and within systems.

1.6 Critical Discussion

The evidence discussed in this talk shows that this active code is required for software development, but it is not enough for Software Engineering success. A software component can function as it was designed, but the entire project can fail due to misunderstanding of the stakeholder needs, requirements not being communicated well, scope going out of management, not enough resources, poor quality control, or engineering risks are not dealt with. Project outcomes are determined by human and engineering factors long before they are implemented, as shown by the importance of requirements misinterpretation and miscommunication as a driver of software-system failure by (Hasnain, et al., 2023).

Failure of software, therefore, should be considered as a possible manifestation of a combination of decisions and situations, not a single coding mistake. Failing software-project case studies revealed recurrent behaviours in them by (Nizam, 2022) and suggested that the process of project failure can be seen as a sequence of phases and behaviours. Likewise, (Amusuo, et al., 2022) highlight the importance of studying the causes, actions and life cycle of software defects and single out causal-chain analysis as a significant approach in the understanding of failure. The results are contributions to the Software Engineering worldview, where the technical implementation needs to be addressed together with the organizational and engineering context in which the software is created.

The Mars Climate Orbiter (MCO) is a good example of the significance of this view. The unit mismatch was of little consequence compared to the eventual outcome; however, interface, communication, verification, and engineering control issues played a role in the problem spreading to a mission failure. The challenge is not whether individual pieces of software work or not, but whether the entire engineered system meets stakeholder needs and requirements and works within reasonable constraints and tolerances of quality and risk. Successful Software Engineering, in turn, relies on ongoing assessment of the trade-offs and downstream effects of software engineering decisions taken throughout the software lifecycle.

1.7 Conclusion

Writing working software programming code is significantly less than writing working software development. Implementation is an essential component of programming, but Software Engineering refers to the broader circumstances necessary for a software system to meet stakeholder requirements and continue to be a success over its lifetime.

Requirements, scope, schedule, cost and resources, quality, and engineering risk are interdependent, and a change or decision in any of these can have consequences and/or trade-offs in others. This wider view aligns with Software Engineering advice to view project management, quality, requirements, risk and lifecycle activities as part of the engineering process not just programming.

The Mars Climate Orbiter case shows the possible ramifications of not managing these interactions properly: A relatively small software-interface discrepancy could propagate in the wake of a series of engineering-control shortcomings. Finally, software engineering is successful not just because it produces working code, but because the whole software solution satisfies its intended needs within reasonable bounds of constraint, is of acceptable quality, and is not too risky.

QUESTION 2 – THINKING AHEAD: DECISIONS ACROSS THE SOFTWARE LIFECYCLE

2.1 Introduction

There are many different critical software project lifecycle decisions/concerns to choose from that should be addressed earlier rather than later. However, all of those choices relate to selecting the appropriate technology stack and system architecture, proper deployment strategy, testing, communication, and security (i.vikash, 2024). Ignoring the degree of importance these overarching communal aspects possess may constrain later choices, increase cost/risk and potentially cause rework. One can observe the importance of the decision sequence by taking a quick look at how delayed decision-making affects software development projects in different SDLC models as an introductory overview.

2.2 Model Reference Introductory Examples

2.2.1 Snapshot of Waterfall Model:

Briefly consider the Waterfall, Agile, and Incremental models. If a software project made use of the Waterfall model, the team involved with carrying out the project would have to keep in mind that Waterfall model cascades upon itself easily as this Software Development Life Cycle (SDLC) has steps with deliverables that tightly depend on one another to continue to the next step (Shriram K. Vasudevan et al., 2018). For example, if poor decisions are made in the Design phase, then those negative consequences will only be noticed when it is too late in the process; appearing only when reaching the Testing or Maintenance phase, and recall there is no cyclic backtracking in the Waterfall model to reverse the consequences (Shriram K. Vasudevan et al., 2018). Consequently, as a result, if any drastic alterations must happen to the project there will be a major increase in cost as a result since essentially the whole project will have to be reworked due to the tight dependency of its process steps.

2.2.2 Snapshot of Agile Model:

Even if a team considers making use of an iterative model that is more flexible, like Agile, there can still arise severe negative consequences from addressing certain concerns later rather than earlier when it matters most (Microsoft Power Platform, n.d.). Software Engineers using an Agile model have to make important decisions before implementation. Thus, unexpected repercussions are only noticed once testing, review or user feedback occurs. Luckily, it is an iterative model, so there is allowable opportunity for improvement in future iterations. However, the damage that occurred is not completely reversible since money was already spent on a drafted project that was not satisfactory to the client. Rework will be required and there may potentially also be more limiting final solutions to correct issues in the project.

2.2.3 Snapshot of Incremental Model:

Now take the Incremental model into account to see if it is more immune to the undesirable outcomes that arise from making important software engineering decisions too late compared to the Agile model. The Incremental model builds software in even smaller chunks than what Agile does. Software engineers have to make important decisions about an increment before implementing it, therefore problems may only show up during later increments after integration with the existing project. Thus, it can be observed that the Incremental model does not go unaffected by deferred software engineering decisions. Although no specific SDLC model will be further inspected regarding the six decision/concerns, recall that all of the SDLC models derive from a general SDLC (Requirement Gathering Phase -> Analysis -> Design -> Coding -> Testing -> Deployment -> Maintenance).

2.3 Significant Software Engineering Decisions and/or Concerns

2.3.1 Decision/Concern 1: Microservice or Monolithic Architecture

What the decision or concern is?

Determining which architecture to use early on is essential for the sake of preventing disorder, confusion, and miscommunication early in the project. Versions of Monolithic and Microservices are the most common architectures used architecture used for software development projects (Geeks For Geeks, 2026).

When it should first be considered?

This decision should be of serious concern starting right from the start at the Requirements Gathering Phase.

When it is likely to be implemented or become operationally significant?

It is likely to become operationally significant at the Design Phase in SDLC.

What information is needed to make a sound decision?

First and foremost, it is crucial that software developers understand the requirements of the client and keep a query tracker document for future requirements communication needs that may arise (Shriram K. Vasudevan, 2018). It is also essential to determine what central architecture the project will adopt as well as identify any major components that will influence each other due to dependencies or tight interactions (Microsoft Power Platform, n.d.). Communication responsibilities must be made clear. Detailed system design is non-negotiable to buffer against potential issues that may arise in the future. Developing detailed prototypes will also be highly advantageous to discover potential future issues (Microsoft Power Platform, n.d.).

Which later activities it enables, constrains or affects?

Not properly determining which architecture to use will make the project lack direction, complicate communication and overall affect the whole project negatively as the architecture is the backbone of the whole project, without proper identification there is no solid foundation and the whole project, developers, and client will suffer.

What could happen if consideration is postponed until implementation time?

If consideration is postponed until implementation time, it will be a major struggle to integrate the project as a whole. There will be a massive amount of rework needed or even involve re-creating the project from scratch which will not sit well with the client financially.

Which credible evidence supports the analysis?

Sam Newman mentions that it is essential to have a clear goal to know where to start a software project, he also states that your project has no direction without an overarching backbone used to guide your way in developing a project. He also makes a comment on how realistic it is to not strictly follow a specific architecture, so drifting slightly to another architecture is perfectly fine, he specifically points out the benefit of a monolith merging with

microservices architecture, as long as a strong foundation is present to start a software project direction (Newman, 2021).

2.3.2 Decision/Concern 2: Non-Functional Requirements – Including Cost Along with Scalability, Latency, Reliability, etc.

What the decision or concern is?

Setting up the correct contracts, like a Service Level Agreement (SLA), and other documentation, like Service Level Objectives (SLO), to keep both the software developers and client accountable for their roles in doing business with one another. Consequences for breaking contract must be clearly stipulated and enforced to create a quality product.

When it should first be considered?

Right at the start of the project during the Requirements Gathering Phase.

When it is likely to be implemented or become operationally significant?

During the Testing, Deployment, and Maintenance Phases.

What information is needed to make a sound decision?

Once again, developers must clearly understand the requirements of the client and keep a query tracker document for future requirements communication needs that may arise (Shriram K. Vasudevan, 2018). The amount and degree of user traffic must be known to deliver expected performance. Client expected product uptime must be established from the start, as well as expected recover time after failures in the system. Expected platform behaviour referring to scalability needs must be understood. Costs surrounding infrastructure, hardware specs, and the cost and consequences of not meeting requirements must be fully decided and rigidly enforced.

Which later activities it enables, constrains or affects?

Properly accounting for non-functional requirements in a timely manner allows developers and architects to make better technical decisions when it comes to creating caching layers and incorporating the right amount of load balancers. Doing the necessary preparation and planning earlier also allows for the testing process to run smoother later on. More importantly, making wiser decisions earlier in this case allows the developing team to provide accurate project cost values while in the design phase, before implementation.

What could happen if consideration is postponed until implementation time?

Drastic failures can lead to complete system outages after deployment and during product scaling. Besides giving the developers rework, the negative effects can also increase costs for the client, cause the client to perceive the product as too much of a risk to maintain; potentially losing the interest and business of the client in the end.

Which credible evidence supports the analysis?

DeLoach & Yurista (2024) from Amazon Web Services (AWS) highlight the importance of including cost as a non-functional requirement to account for along with all the other non-functional requirements like scalability, latency, reliability, etc. They strongly suggest software developers to consider the effects of cost upfront early as well as continuously; during the entire duration of design, development, and operation, as a precursor to achieving all the other non-functional requirements.

2.3.3 Decision/Concern 3: Migratable Deployment Setup

What the decision or concern is?

Having a well laid out deployment plan should be considered earlier rather than later to prevent a final product from becoming both unpresentable and unusable. If the project ends up being usable but also impractical then there is a high chance that it will also be unmigratable, which will in the end make the product useless.

When it should first be considered?

Design and early Deployment Phase

When it is likely to be implemented or become operationally significant?

Deployment and Maintenance Phases

What information is needed to make a sound decision?

To make sensible choices the following must be thoroughly researched and realized: Cloud provider choices, vendor lock-in risks, and compliance regulations. Environment parity needs must also be known so that the product works similarly and consistently across different Personal Computer (PC) environments and Operating Systems (OS's). Application runtime requirements must also be defined to store data, handle scaling, and manage failures effectively.

Which later activities it enables, constrains or affects?

Setting up the deployment strategy and structure appropriately allows for continuous deployment, automated testing environments, and disaster recovery. An apt deployment plan also constrains runtime execution environment and local developer setup to make deployment as continuous as possible.

What could happen if consideration is postponed until implementation time?

A lot of redundant and repetitive rework will pop up due to developer's personal PC setups, which wastes time that they could have spent developing new features and improving the product. The developers will spend more time correcting bugs that result due to their own environment setup and not working on the more important bugs that will get the product closer to being finished. Cost will likely also increase for the client to pay for longer project time development.

Which credible evidence supports the analysis?

Bertram (2020) states that to make sound decisions regarding deployment the developing company must first have a firm grasp on a few very important details. More specifically he refers to being thoroughly acquainted with the platforms that the software will be deployed from, the OS's the software will be running on, available cloud provider choices, and supported hardware needed for deployment. Bertram (2020) also highlights the importance of testing during the deployment process.

2.3.4 Decision/Concern 4: Method of Database Integration

What the decision or concern is?

Selecting the primary data storage model (i.e. Time-Series, relational SQL), selecting the appropriate data storage structure (i.e. relational or non-relational), and establishing core data boundaries and relationships.

When it should first be considered?

Requirements Gathering, Analysis, and Design Phases

When it is likely to be implemented or become operationally significant?

Testing, Deployment, and Maintenance Phases

What information is needed to make a sound decision?

In terms of data, a thorough understanding of what data structure complexity the company is dealing with, and how entity relationships and ACID compliance should be established and incorporated. In terms of accessibility concerns, read/write access patterns should be calibrated appropriately, and query complexity should be observed. Data retention policies and regulatory compliance needs must also be observed and practiced.

Which later activities it enables, constrains or affects?

Providing appropriate database integration allows for efficient feature development, query optimization, as well as dynamic scaling.

What could happen if consideration is postponed until implementation time?

Consequences due to delayed consideration can include massive rework (refactoring data-access layers) and increased risk (in terms of losing valuable client data). Other negative effects can include: complex data migration, possible downtime, and serious performance issues.

Which credible evidence supports the analysis?

Jonker and Krantz discuss several different data integration techniques. Two of the techniques they discuss are Extract, Transform, Load (ETL) and Extract, Load, Transform (ELT), although these integration methods share similarities, they are better suited for different scenarios. ELT works better for a big data project where real-time processing is critical and ETL works better for security compliance, this highlights the importance of thoroughly evaluating what database method companies should make use of before they just pick one that seems the most popular at the time and also most similar to other methods to cover their bases using a holistic approach without doing proper research to meet their specific needs (Jonker & Krantz, n.d.).

2.3.5 Decision/Concern 5: Audit Logging and Monitoring Strategy

What the decision or concern is?

Standardizing a log of the project elements and metrics as well as distributed tracing.

When it should first be considered?

Design Phase

When it is likely to be implemented or become operationally significant?

Testing, Deployment, and Maintenance Phases

What information is needed to make a sound decision?

One set of information needed are the diagnostic needs for distributed requests, another is the audit trail retention requirements for security and compliance.

Which later activities it enables, constrains or affects?

It allows for quick incident response, timely alerts, performance bottleneck identifications, and compliance auditing. It will also constrain API middleware design.

What could happen if consideration is postponed until implementation time?

Without audit logging and monitoring there could result lengthy downtimes of the system and massive rework.

Which credible evidence supports the analysis?

According to Microsoft, audit logging and monitoring strategies are very important for software engineering projects as they provide security measures by detecting unauthorized activities while also provide accountability of the company members building the software product for a client (Microsoft Ignite, 2026). Without it there is no way of knowing which company members were responsible for what roles in the software development process, the location of the tasks performed during development as well as the outcomes of the tasks accomplished on the project.

2.3.6 Decision/Concern 6: Authentication, Authorization, and Identity Concerns

What the decision or concern is?

Designing resilient security architecture for user authentication, robust access-control, efficient single sign on (SSO), as well as fitting identity provider integration.

When it should first be considered?

Requirements Gathering and Design Phases

When it is likely to be implemented or become operationally significant?

Testing, Deployment, and Maintenance Phases

What information is needed to make a sound decision?

The following are required to be given to the software developing company: User personas, privilege hierarchies, multi-tenancy requirements. The developing company should also be developing according to regulatory compliance standards as well as federal requirements.

Which later activities it enables, constrains or affects?

It allows secure API design, feature gating, and enterprise client onboarding. It constrains domain logic implementation, route guarding, and API payload definitions.

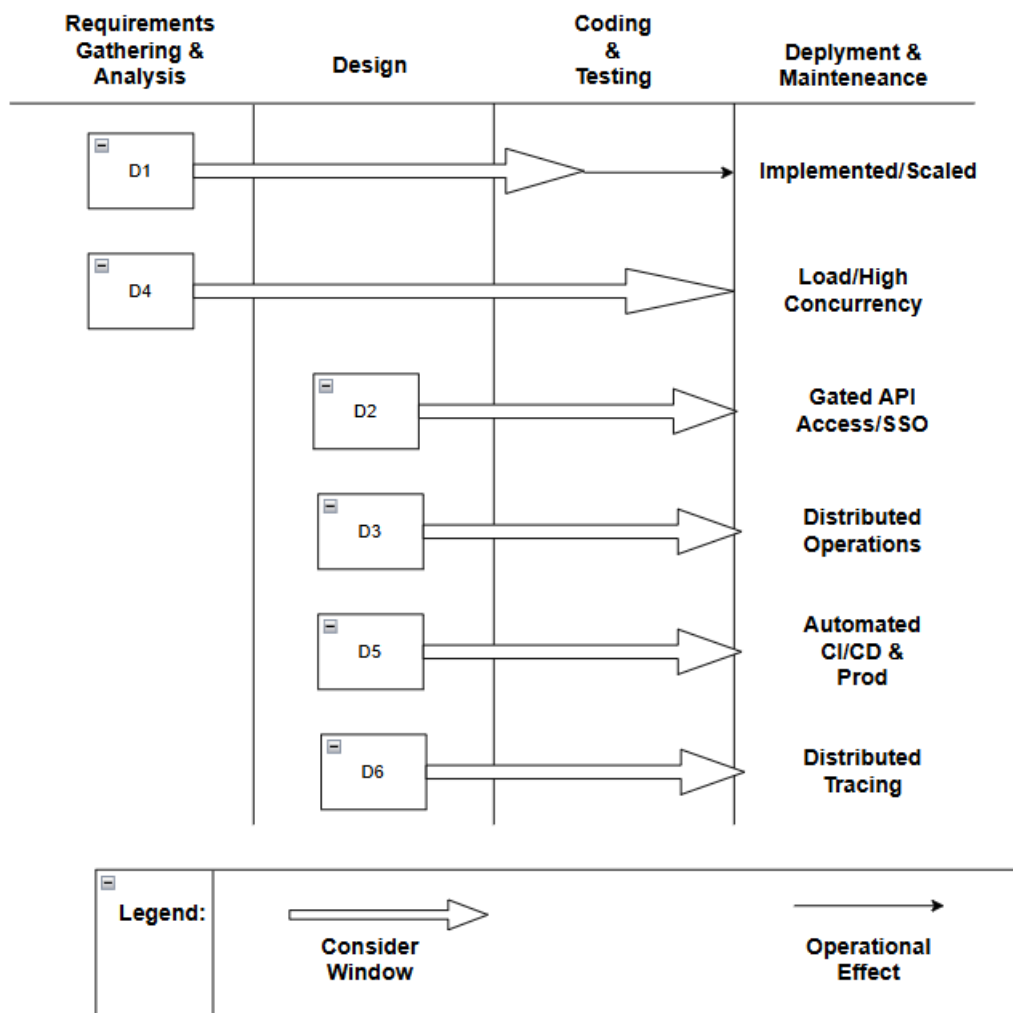
What could happen if consideration is postponed until implementation time?

Security vulnerabilities could be introduced. Frontend-backend communication contracts could also be broken. This can cause rework, significant security risk to the client, reputation risk to the developing company, as well as paying penalty costs.

Which credible evidence supports the analysis?

Security is a rapidly growing concern for everyone, but for big tech companies that are clients it is of utmost importance as they have clients that depend on their services as well. Identity and access management (IAM) handles the provision and protection of digital identities by making sure “the right people can access the right resources for the right reasons at the right time” (Forrest & Kosinski, n.d.).

2.4 Decision Horizon Map



Decisions:

D1 Method of Database Integration

D2 Authentication, Authorization, IAM

D3 Microservices or Monolithic Architecture

D4 Non Functional Requirements (Including Cost)

D5 Migration Deployment Setup

D6 Audit Logging & Monitoring

QUESTION 3 – EVIDENCE-BASED TECHNOLOGY STACK DECISIONS

3.1 Introduction

Selecting a technology stack is an engineering decision rather than simply choosing the most popular programming language or framework. A suitable stack must align with the capability of the development team, security requirements, maintainability expectations, available testing tools, deployment environment, cost constraints and the expected evolution of the system.

Research into modern technology and data stacks supports this context-based approach. Tlemcani, Faroukhi and Gahi identify factors including scalability, cost efficiency, vendor neutrality, learning curve and business impact as relevant when evaluating modern technology components. Their systematic review further shows that technology adoption should reflect organisational maturity rather than assuming that one architecture is universally suitable.

Stocker and Wahler also demonstrate that apparently routine changes to software-stack components can create measurable downstream effects. Their empirical study of Spring Boot and Java Virtual Machine versions found substantial differences in application energy consumption between stack configurations. This supports the principle that technology choices should be evaluated using evidence rather than assumptions that a newer version is automatically better.

For this comparison, two viable contemporary web application stacks are evaluated:

Stack A — Java-based web stack

- React frontend
- Spring Boot backend
- Java/JVM runtime
- PostgreSQL database
- Docker/container-based deployment

Stack B — .NET-based web stack

- React frontend
- ASP.NET Core backend

- C#/.NET runtime
- PostgreSQL database
- Docker/container-based deployment

Using the same frontend, database and general deployment approach allows the comparison to focus primarily on differences in the server-side framework and runtime rather than comparing two completely unrelated system designs.

3.2 Overview of the Two Alternatives

Stack A: React + Spring Boot + PostgreSQL

Spring Boot is a Java-based framework designed for stand-alone, production-grade applications. Its official documentation includes built-in support and integrations for areas such as security, metrics, health monitoring and externalised configuration. Spring Boot applications can also be packaged as executable applications or container images.

The Java/JVM ecosystem also provides an interesting lifecycle consideration. Stocker and Wahler found that changing Spring Boot and JVM versions could materially affect the energy consumption of the same application. Newer JVM releases and virtual threads produced energy improvements in their case study without requiring major application-level changes.

Stack B: React + ASP.NET Core + PostgreSQL

ASP.NET Core is Microsoft's cross-platform framework for developing contemporary web applications and APIs using C# and .NET. Its official security documentation includes support for authentication, authorisation, data protection, HTTPS enforcement, secure handling of development secrets, Cross-Site Request Forgery protection and Cross-Origin Resource Sharing configuration.

ASP.NET Core can also run in Docker containers and on Linux-based environments, meaning it is not restricted to Windows deployment.

Both stacks therefore represent viable contemporary alternatives rather than one deliberately weak and one deliberately strong option.

3.3 Comparative Analysis

Criterion	Spring Boot / Java Stack	ASP.NET Core / .NET Stack
Team capability and learning curve	Requires knowledge of Java, Spring conventions, dependency injection and the wider Spring ecosystem. Spring Boot attempts to reduce configuration through opinionated defaults.	Requires knowledge of C#, .NET and ASP.NET conventions. Strong IDE and CLI integration can support development. Actual learning difficulty depends heavily on existing team experience.
Security ecosystem	Spring Security provides authentication, authorisation	ASP.NET Core provides integrated mechanisms for

	and security integration. Spring Boot web applications using Spring Security are secured by default, although production configuration remains the developer's responsibility.	authentication, authorisation, data protection, HTTPS, CSRF protection, CORS and other web-security concerns.
Maintainability	Strong separation of concerns and dependency injection can support maintainable applications, although dependency and framework upgrades create lifecycle responsibilities.	Strong typing, dependency injection and established application patterns can support maintainability. .NET version and package upgrades must similarly be managed over the system lifecycle.
Testing and tooling	Spring Boot includes dedicated testing modules and commonly uses JUnit, AssertJ, Hamcrest and integration-testing facilities.	.NET supports unit, integration and load testing, with frameworks including MSTest, NUnit and xUnit and integration with CLI tools and IDEs.
Deployment/platform compatibility	Applications can run as Java applications, on physical or virtual servers, cloud platforms and Docker-compatible containers.	ASP.NET Core supports Linux, containerised deployment and different hosting environments in addition to Microsoft platforms.
Development and operational cost	Core technologies are open source, but labour, hosting, infrastructure and maintenance remain significant costs. Runtime configuration may also affect resource and energy consumption.	Core .NET technologies are also open source; operational costs depend more on hosting architecture, developer effort and supporting services than simply the framework license.
Future change and maintenance	Spring/JVM upgrades can improve functionality and performance but may also produce unexpected operational effects, making benchmarking and controlled upgrading important.	Strong current testing and deployment tooling supports controlled evolution, but framework/runtime upgrades still require testing and dependency management.

Spring Boot provides extensive testing support through **spring-boot-test** and **spring-boot-test-autoconfigure**, while the standard starter incorporates commonly used testing libraries. .NET similarly provides formal support for unit, integration and load testing and supports frameworks such as MSTest, NUnit and xUnit through its development tooling and CLI.

The deployment comparison is also relatively close. Spring Boot can generate Docker-compatible container images and can be deployed across cloud platforms and physical or virtual machines. Microsoft likewise documents Docker-based ASP.NET Core deployment

and Linux hosting, demonstrating that modern .NET is not dependent on Windows-only infrastructure.

Consequently, the comparison does not reveal a universally dominant stack. Instead, different project conditions can cause relatively similar technologies to be evaluated differently.

3.4 Weighted Decision Matrix

A five-point scoring system is used:

1 = very weak, 2 = weak, 3 = acceptable, 4 = strong, 5 = very strong.

The matrix assumes a hypothetical small-to-medium development team building a long-lived web information system with normal security requirements, container-based deployment and no pre-existing organisational commitment to either Java or .NET. This is an **evaluation scenario only and is not a CivicConnect technology decision**.

Criterion	Weight	Spring Score /5	Spring Weighted	.NET Score /5	.NET Weighted
Team capability & learning curve	15%	4	0.60	4	0.60
Security ecosystem	20%	5	1.00	5	1.00
Maintainability	15%	4	0.60	5	0.75
Testing & tooling	10%	5	0.50	5	0.50
Deployment compatibility	10%	5	0.50	5	0.50
Development & operational cost	15%	4	0.60	4	0.60
Future change & maintenance	15%	4	0.60	5	0.75
TOTAL	100%		4.40 / 5		4.70 / 5

3.5 Justification of the Weights

Security — 20%. Security receives the highest weighting because a web-based information system may expose application functionality and organisational data to external networks. Security failures can create consequences far beyond development inconvenience. Both frameworks therefore benefit from mature security ecosystems. Spring Security provides integrated web security capabilities, while ASP.NET Core provides mechanisms covering areas including authentication, authorisation, HTTPS, data protection, CSRF and CORS.

Team capability and learning curve — 15%. A technically capable framework can still be unsuitable if the development team cannot use it effectively. Tlemcani et al. explicitly include learning curve among their technology evaluation criteria, supporting its inclusion in the matrix.

Maintainability — 15%. Information systems usually continue changing after initial deployment. Maintainability therefore affects long-term cost, defect risk and the team's ability to introduce new requirements.

Development and operational cost — 15%. Cost is broader than software licensing. It includes developer time, infrastructure, cloud resources, maintenance and upgrades. The systematic review by Tlemcani et al. specifically considers cost efficiency alongside technical suitability.

The Stocker and Wahler study also demonstrates why operational cost should not be viewed purely as a hosting-price calculation. Different versions of software-stack components can consume significantly different amounts of energy under comparable workloads.

Future change and maintenance — 15%. Stack decisions create long-term consequences because runtimes, dependencies and frameworks evolve. Stocker and Wahler's study is particularly relevant here: upgrading components can produce both improvements and unexpected regressions, demonstrating why upgrade decisions should be benchmarked rather than blindly accepted.

Testing and tooling — 10%. Testing is essential for controlling defects and supporting future modifications, but its weighting is slightly lower because both alternatives already provide mature testing ecosystems.

Deployment/platform compatibility — 10%. Both alternatives support contemporary containerised and cross-platform deployment, so this criterion is important but is less likely to distinguish the two stacks strongly in this particular comparison.

3.6 Evidence-Based Interpretation of the Matrix

The matrix gives the ASP.NET Core stack a small advantage, mainly because of its integrated tooling and strong long-term development ecosystem under the assumptions used in this comparison. However, the difference should not be interpreted as an absolute technical ranking.

This is consistent with the research by Tlemcani et al. Their systematic review does not evaluate architecture independently of context; instead, components are considered according to factors such as scalability, cost efficiency, vendor neutrality, learning curve and business impact and across different organisational archetypes. The authors also propose staged adoption based on organisational maturity.

This is important because a technology that receives a score of five for one organisation could reasonably receive three for another.

For example, if an organisation already employed experienced Java/Spring developers and operated substantial JVM infrastructure, the **team capability**, **maintenance** and possibly **cost** ratings for the Spring stack would increase. The matrix could then favour Spring without either matrix being mathematically incorrect.

3.7 Limitations of the Weighted Decision Matrix

Although a weighted decision matrix improves transparency, it has several limitations.

First, the weighting system involves **human judgement**. Giving security 20% rather than 15% reflects the priorities of the hypothetical project. A different project could legitimately assign more importance to cost, development speed or deployment restrictions.

Second, scoring can create a false appearance of precision. A score of five rather than four does not necessarily mean that one framework is precisely 25% better in that area.

Third, technology capability changes over time. Framework versions, tooling, security support and deployment services continually evolve. The energy study demonstrates this particularly well: even moving between versions of the same JVM or framework can change measurable system behaviour.

Fourth, the matrix cannot fully represent organisational maturity. The systematic review by Tlemcani et al. found that adoption success depends on more than individual technologies and highlights implementation maturity and governance as significant considerations. Governance was identified as an especially consequential area for long-term adoption.

Therefore, a matrix should be treated as a **decision-support mechanism**, not an automatic decision-maker.

3.8 Critical Question

Why is the highest-scoring or most popular technology stack not automatically the correct technology for every software project?

The highest-scoring stack is not automatically correct because **software engineering decisions are context-dependent**.

A technology may be widely adopted while still being inappropriate for a particular project's team capability, infrastructure, security requirements, financial constraints or maintenance strategy. Tlemcani et al.'s research supports this principle by evaluating technologies according to several criteria and different organisational contexts instead of presenting one universally superior architecture.

The highest score in a decision matrix also depends on the weights assigned to the criteria. If a project prioritises developer familiarity and already has a Java engineering team, Spring Boot could outperform .NET. If another organisation has extensive C# knowledge and existing .NET infrastructure, ASP.NET Core could provide lower risk and maintenance cost.

Furthermore, apparently small stack choices may create downstream lifecycle consequences. Stocker and Wahler demonstrated that changing Spring Boot and JVM versions can significantly affect energy consumption, including unexpected regressions. Therefore, popularity or recency cannot replace empirical evaluation.

The engineering objective should consequently be to select the stack that provides the **best fit between technical capabilities, organisational constraints and lifecycle requirements**, rather than selecting whichever technology receives the most attention or the largest numerical score.

QUESTION 4 – RESPONSIBLE AI IN SOFTWARE ENGINEERING

4.1 Introduction

Generative Artificial Intelligence (AI) can support several activities within contemporary Software Engineering, including requirements analysis, software design, coding, testing, review and documentation. However, the use of AI does not remove the need for professional engineering judgement. Responsible use requires AI-generated outputs to be governed, verified and controlled throughout the software lifecycle.

Lu et al. (2022b) argue that responsible AI cannot be achieved through high-level ethical principles alone because these principles are often difficult to translate into practical engineering activities. They propose an approach based on three connected areas: multi-level governance, responsible development processes and responsible-AI-by-design techniques. Similarly, Sanderson et al. (2022) identify accountability, reliability, transparency, explainability, privacy and fairness as important considerations that need to be incorporated into software development rather than considered only after deployment.

Therefore, generative AI should be treated as an **engineering assistant**, rather than an independent engineering authority.

4.2 Three Valuable Uses of Generative AI

1. Requirements analysis and documentation

Generative AI can assist engineers during requirements activities by summarising stakeholder information, identifying ambiguous statements, proposing user stories and producing draft acceptance criteria. It can also help reorganise large amounts of requirements information into a more consistent structure.

This could reduce repetitive documentation work and help engineers identify areas requiring further discussion. However, generated requirements cannot simply be accepted as correct. Requirements must remain connected to the actual needs of stakeholders.

Lu et al. (2022b) emphasise that requirements should be gathered from appropriate stakeholders, specified in a measurable manner where possible and maintained with traceability into design, code and test cases. They also warn that responsible-AI requirements are often expressed only as vague objectives instead of verifiable system

requirements. Generative AI can therefore **assist the requirements process**, but stakeholder validation remains necessary.

2. Coding and software design assistance

Generative AI can support software developers by producing boilerplate code, suggesting alternative implementations, explaining unfamiliar code, assisting with refactoring and proposing possible software designs.

Its main value is not that the generated answer must be correct, but that it can provide a starting point that an engineer can evaluate and improve. This may be particularly useful when performing repetitive programming activities or investigating unfamiliar APIs and design alternatives.

However, software engineering involves more than producing functioning code. Lu et al. (2022b) argue that responsible AI must consider system-level architecture and design rather than focusing only on individual algorithms. Their roadmap therefore promotes a responsible-AI-by-design approach in which quality and ethical requirements influence architectural and implementation decisions.

For generative-AI-assisted coding, this means generated code should be treated as a **candidate implementation** that remains subject to the same architectural, security and quality requirements as code written manually.

3. Testing, review and technical documentation

Generative AI can also assist with the generation of candidate unit tests, edge cases, review checklists and technical documentation. For example, an engineer could provide a function and ask an AI system to suggest unusual input combinations that should be tested.

This can support broader consideration of possible failures, but AI-generated tests cannot prove that a system is correct. Engineers still need to determine whether the tests accurately reflect the requirements and whether important scenarios have been omitted.

Lu et al. (2022a) found that practitioners wanted stronger **continuous monitoring and validation** of AI systems rather than one-time ethical assessments. Their study also emphasises traceability and provenance, including version control and records that make it possible to determine how software artefacts were produced and changed.

Generative AI can therefore increase the amount of material available for testing and review, while responsibility for deciding whether that material is complete and correct remains with the engineering team.

4.3 Three Significant Risks and Limitations

Risk 1 — Incorrect or unreliable output

Generative AI can produce output that appears technically convincing while containing incorrect assumptions, invalid APIs, faulty logic or inappropriate engineering recommendations. If developers trust the output because it is expressed confidently, defects may be introduced into the software.

Reliability is one of the major concerns identified by Sanderson et al. (2022), together with safety, accountability and transparency. The authors argue that responsible AI principles need to be converted into concrete engineering practices rather than treated as general statements of intent.

The consequence in software development could therefore be:

Unverified AI output → incorrect implementation → software defect or system failure.

The appropriate control is independent human review followed by technical testing.

Risk 2 — Security, privacy and bias

AI-assisted engineering can introduce security and privacy risks. Generated code may contain insecure patterns or fail to consider important security requirements. Privacy problems may also arise when developers provide confidential source code, personal information, credentials or sensitive organisational information to an external AI service.

Responsible AI also requires consideration of potential unfairness or bias, especially when generated material influences decisions affecting people.

Lu et al. (2022b) identify **privacy, safety and security, fairness and non-discrimination** as major responsible-AI principles that need to be considered across the lifecycle. They additionally argue that these concerns should be considered as engineering requirements rather than issues addressed only after a system has been implemented.

Lu et al. (2024b) similarly argue that ethical issues can occur throughout the development lifecycle and across both AI and non-AI system components. Their Responsible AI Pattern Catalogue therefore separates responsible-AI practices into governance, development-process and product-level patterns.

The consequence can be represented as:

Poor AI usage controls → insecure, private or biased output → technical, ethical or organisational harm.

Risk 3 — Over-reliance and loss of accountability

A further risk occurs when engineers begin treating AI-generated recommendations as authoritative. This can create **automation bias**, where an engineer accepts an answer because the AI produced it rather than because sufficient engineering evidence supports it.

This can also damage traceability. If a development team cannot explain where an engineering decision came from, who reviewed it and why it was accepted, later audits and defect investigations become more difficult.

Lu et al. (2022a) found that traceability, provenance, version control and immutable records can contribute to accountability because they provide evidence of how system artefacts evolved.

The Responsible AI Pattern Catalogue further proposes governance, process and product patterns designed to translate high-level ethical principles into actionable controls across the lifecycle (Lu et al., 2024b).

Therefore:

Blind dependence on AI → reduced human judgement and traceability → weakened accountability.

4.4 Human Verification and Control Process

Material AI-generated output should not move directly from an AI system into a controlled software product. A human-centred verification process should occur first.

Stage	Human verification/control
1. Record the AI contribution	Identify which material was AI-assisted and preserve relevant provenance where appropriate.
2. Requirements verification	Compare the output against approved requirements, stakeholder needs and system constraints.
3. Technical review	A competent engineer examines the generated code, design, test or documentation for correctness and maintainability.
4. Security and responsible-AI review	Check security, privacy, potential bias, explainability and other applicable risks.
5. Independent evidence	Important technical claims should be checked against documentation, standards, research or other credible evidence.
6. Testing	Execute appropriate unit, integration, system, regression and security testing rather than assuming generated material is correct.
7. Peer approval and controlled integration	A responsible human approves material changes before they enter the controlled repository or product.

8. Monitoring after deployment	Material components should continue to be monitored where changes in behaviour or operating conditions could create new risks.
---------------------------------------	--

This approach reflects the lifecycle-oriented view proposed by Lu et al. (2022b), where governance, development processes and system design must work together rather than treating responsible AI as a single final review.

It also addresses a significant weakness identified by Lu et al. (2022a): one-time risk assessments are inadequate for systems whose behaviour or operating context can change. Their empirical study therefore highlights the importance of continuous validation as well as traceability between data, models, code and configuration.

Lu et al. (2024a) similarly describe responsible AI as a lifecycle concern involving governance mechanisms, engineering practices, architecture and design patterns, and system-level techniques rather than a single technical control.

4.5 Accountability

If an engineer accepts AI-generated output that later introduces a defect, vulnerability or incorrect engineering decision, who remains accountable and why?

Human engineers and the responsible organisation remain accountable for the accepted engineering decision. Responsibility cannot simply be transferred to the generative AI system.

The engineer who accepts material AI-generated output has made a human engineering decision to incorporate that output into the product. The engineer is therefore responsible for performing appropriate verification, review and testing before acceptance. At the same time, accountability should not be understood as belonging only to one developer. The organisation also has responsibility for establishing appropriate governance, policies, review procedures and controls around the use of AI.

Lu et al. (2022b) explicitly identify accountability as requiring that those responsible for different phases of an AI system's lifecycle can be identified and held accountable for system outcomes. They also advocate role-level accountability and lifecycle traceability.

This principle is reinforced by Lu et al. (2024b), whose Responsible AI Pattern Catalogue distributes responsibility across governance, development processes and product design instead of assuming that responsible AI can be achieved by the AI technology itself. Sanderson et al. (2022) similarly treat accountability as one of the central ethical concerns that must be operationalised during system development and operation.

Therefore, if an AI-generated code fragment later creates a security vulnerability, saying “**the AI wrote it**” does not remove accountability. The professional responsibility lies with the people and organisation that decided to review, approve, integrate and deploy that output.

A useful engineering relationship is:

AI generates → human verifies → human approves → organisation deploys → human/organisational accountability remains.

4.6 Conclusion

Generative AI can provide meaningful engineering value by assisting requirements work, coding and design, and software testing and documentation. However, these benefits should not result in unquestioned reliance on generated output. Incorrect results, security and privacy concerns, bias and weakened accountability can introduce significant downstream consequences.

The research consistently demonstrates that responsible AI requires **governance, lifecycle controls, traceability, verification and identifiable human responsibility** rather than reliance on high-level ethical principles alone (Lu et al., 2022a; Lu et al., 2022b; Sanderson et al., 2022; Lu et al., 2024b).

Generative AI should therefore augment professional engineering judgement rather than replace it.

REFERENCE LIST

Bibliography

Amusuo, P. C. et al., 2022. *Reflections on software failure analysis*. s.l., s.n., pp. 1615-1620.

Bertram, A., 2020. *Write a software deployment plan under these guidelines*. [Online]
Available at: <https://www.techtarget.com/it-infrastructure/tip/Write-a-software-deployment->

plan-under-these-guidelines

[Accessed 3 Sept 2026].

Forrest, M. K. & A., n.d.. *What is IAM?*. [Online]

Available at: <https://www.ibm.com/think/topics/identity-access-management>

[Accessed 3 Sept 2026].

Geeks For Geeks, 2026. *Monolithic Vs Microservices Architecture*. [Online]

Available at: <https://www.geeksforgeeks.org/software-engineering/monolithic-vs-microservices-architecture/>

[Accessed 3 Sept 2026].

Hasnain, M. et al., 2023. Empirical analysis of software success rate forecasting during requirement engineering processes. *Computers, Materials and Continua*, 74(1), pp. 783-799.

i.vikash, 2024. *Critical Factors to Consider When Selecting a Tech Stack for Your Next Project (with Examples)*. [Online]

Available at: <https://medium.com/@i.vikash/critical-factors-to-consider-when-selecting-a-tech-stack-for-your-next-project-with-examples-ec7744bbd24a>

[Accessed 2 Sept 2026].

Krantz, A. J. & T., n.d.. *Data integration techniques and methods*. [Online]

Available at: <https://www.ibm.com/think/topics/data-integration-techniques-methods>

[Accessed 3 Sept 2026].

Matranga, E., Fong, C., DeTurris, D. & Flumerfelt, S., 2024. Systems Perspective Outcomes from Aerospace Failure Investigations. *INCOSE International Symposium*, 34(1), pp. 2332-2350.

Microsoft Ignite, 2026. *Audit logging and monitoring overview*. [Online]

Available at: <https://learn.microsoft.com/en-us/compliance/assurance/assurance-audit-logging>

[Accessed 3 Sept 2026].

Microsoft Power Platform, n.d.. *What is the software development life cycle?*. [Online]

Available at: <https://www.microsoft.com/en-us/power-platform/topics/phases-of-the-software-development-lifecycle>

[Accessed 3 Sept 2026].

Newman, S., 2021. Splitting the Monolith. In: *Building Microservices*. s.l.: O'Reilly Media Inc., p. Chapter 3.

Nizam, A., 2022. Software project failure process definition. *IEEE access*, Volume 10, pp. 34428-34441.

Pasham, S. D., 2024. Managing Requirements Volatility in Software Quality Standards: Challenges and Best Practices. *International Journal of Modern Computing*, 7(1), pp. 123-140.

Prause, C. R., Gerlich, R. & Gerlich, R., 2024. Fatal software failures in spaceflight. *Encyclopedia*, 4(2), pp. 936-965.

Prokop, L., 2022. *Software Common Mode Failure Incidence*, s.l.: s.n.

Prokop, L. E., 2024. Software error incident categorizations in aerospace. *Journal of Aerospace Information Systems*, 21(10), pp. 775-789.

Shriram K. Vasudevan, R. M. D. S. V. S. P. R. N., 2018. Software Process Model. In: N/A, ed. *Software Project Management*. Oxford: Alpha Science International Ltd., pp. 3.1-3.21.

Yurista, A. D. & P., 2024. *Achieving Frugal Architecture using the AWS Well-Architected Framework guidance*. [Online]

Available at: <https://aws.amazon.com/blogs/architecture/achieving-frugal-architecture-using-the-aws-well-architected-framework-guidance/>

[Accessed 3 Sept 2026].

Question 2 References

Amusuo, P. C. et al., 2022. *Reflections on software failure analysis*. s.l., s.n., pp. 1615-1620.

Bertram, A., 2020. *Write a software deployment plan under these guidelines*. [Online]

Available at: <https://www.techtarget.com/it-infrastructure/tip/Write-a-software-deployment-plan-under-these-guidelines>

[Accessed 3 Sept 2026].

Forrest, M. K. & A., n.d.. *What is IAM?*. [Online]

Available at: <https://www.ibm.com/think/topics/identity-access-management>

[Accessed 3 Sept 2026].

Geeks For Geeks, 2026. *Monolithic Vs Microservices Architecture*. [Online]

Available at: <https://www.geeksforgeeks.org/software-engineering/monolithic-vs-microservices-architecture/>

[Accessed 3 Sept 2026].

Hasnain, M. et al., 2023. Empirical analysis of software success rate forecasting during requirement engineering processes. *Computers, Materials and Continua*, 74(1), pp. 783-799.

i.vikash, 2024. *Critical Factors to Consider When Selecting a Tech Stack for Your Next Project (with Examples)*. [Online]

Available at: <https://medium.com/@i.vikash/critical-factors-to-consider-when-selecting-a-tech-stack-for-your-next-project-with-examples-ec7744bbd24a>

[Accessed 2 Sept 2026].

Krantz, A. J. & T., n.d.. *Data integration techniques and methods*. [Online]

Available at: <https://www.ibm.com/think/topics/data-integration-techniques-methods>

[Accessed 3 Sept 2026].

Matranga, E., Fong, C., DeTurris, D. & Flumerfelt, S., 2024. Systems Perspective Outcomes from Aerospace Failure Investigations. *INCOSE International Symposium*, 34(1), pp. 2332-2350.

Microsoft Ignite, 2026. *Audit logging and monitoring overview*. [Online]

Available at: <https://learn.microsoft.com/en-us/compliance/assurance/assurance-audit-logging>

[Accessed 3 Sept 2026].

Microsoft Power Platform, n.d.. *What is the software development life cycle?*. [Online]

Available at: <https://www.microsoft.com/en-us/power-platform/topics/phases-of-the-software-development-lifecycle>

[Accessed 3 Sept 2026].

Newman, S., 2021. Splitting the Monolith. In: *Building Microservices*. s.l.:O'Reilly Media Inc., p. Chapter 3.

Nizam, A., 2022. Software project failure process definition. *IEEE access*, Volume 10, pp. 34428-34441.

Pasham, S. D., 2024. Managing Requirements Volatility in Software Quality Standards: Challenges and Best Practices. *International Journal of Modern Computing*, 7(1), pp. 123-140.

Prause, C. R., Gerlich, R. & Gerlich, R., 2024. Fatal software failures in spaceflight. *Encyclopedia*, 4(2), pp. 936-965.

Prokop, L., 2022. *Software Common Mode Failure Incidence*, s.l.: s.n.

Prokop, L. E., 2024. Software error incident categorizations in aerospace. *Journal of Aerospace Information Systems*, 21(10), pp. 775-789.

Shriram K. Vasudevan, R. M. D. S. V. S. P. R. N., 2018. Software Process Model. In: N/A, ed. *Software Project Management*. Oxford: Alpha Science International Ltd., pp. 3.1-3.21.

Yurista, A. D. & P., 2024. *Achieving Frugal Architecture using the AWS Well-Architected Framework guidance*. [Online]
Available at: <https://aws.amazon.com/blogs/architecture/achieving-frugal-architecture-using-the-aws-well-architected-framework-guidance/>
[Accessed 3 Sept 2026].

Question 3 References

Stocker, M. and Wahler, M. (2026) 'Energy-aware decision making in software stack upgrades', *Journal of Systems and Software*, 241, article 112963. doi: 10.1016/j.jss.2026.112963.

Tlemcani, C., Faroukhi, A.Z. and Gahi, Y. (2025) 'Modern Data Stack Demystified: A Strategic Framework for Scalable Adoption', *2025 11th International Conference on Optimization and Applications (ICOA)*. IEEE. doi: 10.1109/ICOA66896.2025.11236951.

Tlemcani, C., Faroukhi, A.Z. and Gahi, Y. (2026) 'From ETL to Modern Data Stack: A Systematic Review and Strategic Implementation Framework', *Information*, 17(7), article 630. doi: 10.3390/info17070630.

Question 4 References

Lu, Q., Zhu, L., Xu, X., Whittle, J., Douglas, D. and Sanderson, C. (2022a) 'Software engineering for responsible AI: An empirical study and operationalised patterns', *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*, pp. 241–242. doi: 10.1145/3510457.3513063.

Lu, Q., Zhu, L., Xu, X., Whittle, J. and Xing, Z. (2022b) 'Towards a roadmap on software engineering for responsible AI', *Proceedings of the 1st International Conference on AI Engineering: Software Engineering for AI*, pp. 101–112. doi: 10.1145/3522664.3528607.

Lu, Q., Zhu, L., Whittle, J. and Xu, X. (2024a) *Responsible AI: Best Practices for Creating Trustworthy AI Systems*. Addison-Wesley Professional.

Lu, Q., Zhu, L., Xu, X., Whittle, J., Zowghi, D. and Jacquet, A. (2024b) 'Responsible AI Pattern Catalogue: A Collection of Best Practices for AI Governance and Engineering', *ACM Computing Surveys*, 56(7), pp. 1–35. doi: 10.1145/3626234.

Sanderson, C., Lu, Q., Douglas, D., Xu, X., Zhu, L. and Whittle, J. (2022) 'Towards implementing responsible AI', *2022 IEEE International Conference on Big Data*, pp. 5076–5081. doi: 10.1109/BigData55660.2022.10021121.

APPENDIX A – AI RESEARCH & VERIFICATION RECORD

Team Member	AI Tool/Use	Purpose	Output Used?	How Independently Verified?	What was changed /rejected /accepted?
578122	Gemini Flash	To Direct Research Scope & To compare/confirm facts obtained from research	Partially Yes & Yes	Compared with other academic/standards sources, recognised professional resources, and official technological documentation to check for alignment	Similar decision/ concerns were altered and merged as well as aligned facts
601611	ChatGpt	To generate an initial assignment outline, structure the workflow, and summarize source materials	Partially	Cross-referenced summaries and outlines against primary academic texts, course materials, and official guidelines	Accepted the general structural framework and key summary points; rejected irrelevant background details and adjusted the outline to better align with the

					assignment criteria
601597	ChatGPT	To assist with outlining and structuring Question 1, guide research, evaluate source relevance, and cross-reference research findings.	Partially – Yes	AI-assisted claims were cross-referenced against academic journal articles, conference papers, official NASA material, and publisher records. Sources were independently located and reviewed before being included.	Accepted the general structure and relevant research directions. Revised wording and arguments to fit the assignment requirements. Weaker or irrelevant sources and unsupported claims were rejected, and factual claims were corrected or refined after cross-referencing with academic and official sources.